

系统功能报告

清华大学程序设计训练 (Python) 2026 暑期课程 · 实验一：爬虫与信息检索

除 CSS 设计和 README 生成以外均使用手工编程实现。

源代码详细信息见源代码文件夹下 README.md。

一、系统概述

基于 Django 6.0 构建的 Web 应用，自动爬取网易云音乐数据，提供歌曲/歌手的浏览、搜索与评论功能。前端使用 Django Templates + 原生 CSS，数据库使用 SQLite3。

二、数据规模

数据类型	数量	存储方式
歌曲	4,718 首	SQLite / Django fixture
歌手	1,380 位	SQLite / Django fixture
评论	50,716 条	SQLite / Django fixture
歌曲封面图	4,718 张	本地 <code>data/images/</code> (约 6.8GB)
歌手封面图	1,380 张	同上
总计图片	6,098 张	同上

原始作业要求为 200 位歌手 / 2,000 首歌曲，实际数据量约为要求的 7 倍。歌手 811「未知歌手」为网易云无歌手页面艺人的合集占位。

三、功能模块

3.1 歌曲列表页 `/songlist`

每日随机排序：以当天日期为随机种子 (`date.today()`)，预生成 10,000 个随机数的映射表。每首歌按其 ID 对应的随机权重排序——同一天内所有人看到相同顺序，隔天自动轮换。

布局：双栏网格，每行两张歌曲卡片。卡片含大号序号 (40px 浅蓝)、72×72 缩略图、24px 加粗蓝色歌名、歌手链接 (灰色，hover 变蓝)。

翻页：每页 20 首，全局连续编号 (翻页后序号接续上一页)。分页组件支持首末页跳转、当前页 ± 2 范围显示 (远处用 ... 省略)、输入页码直接跳转。

3.2 歌手列表页 `/singerlist`

与歌曲列表共用同一套每日随机种子和映射表，每页 25 位，五栏网格布局。歌手卡片为正方形封面图 + 居中加粗歌手名，整卡可点击，hover 浮起阴影。

3.3 歌曲详情页 `/song/<id>`

信息展示：左侧 320×320 封面大图，右侧蓝色竖线分隔。歌名 96px 加粗，下方依次排列歌手链接（用 `/` 分隔，每 8 个换行）和网易云来源链接。

网易云外链播放器：嵌入 iframe (`//music.163.com/outchain/player?type=2&id=<true_id>&auto=1`)，330×86。通过正则从 `source_url` 中提取真实网易云歌曲 ID。

多栏歌词排版：根据歌词行数自动计算栏数 (`ceil(行数/30)`，最多 5 栏)，CSS `column-rule` 栏间竖线分隔，每行 hover 加粗变蓝。无歌词时显示「纯音乐，请欣赏」。

评论区：底部固定毛玻璃半透明输入栏 (`backdrop-filter: blur`)，输入昵称（可选）和评论内容。评论按时间倒序排列，每条左侧 240px 元信息（昵称 22px 蓝 + 日期），分隔竖线，右侧 17px 正文。hover 时显示删除按钮。底部 80px 留白防止遮挡。

3.4 歌手详情页 `/singer/<id>`

左侧 320×320 艺人封面，右侧超大字姓名 + 来源链接 + 简介文本。下方以双栏歌曲卡片形式展示该歌手全部作品（通过 M2M 反向查询 `singer.songs.all()`）。

3.5 搜索页 `/search`

四级优先级排序：歌手名（精确匹配）> 歌手简介 > 歌曲名 > 歌词。使用 Django ORM 的 `Q` 组合条件 + `Case/When` 标注优先级，单次查询 < 1 秒。

Tab 切换：按钮形式切换「歌手·N」/「歌曲·N」，激活态白字蓝底。结果独立分页（歌手 25/页、歌曲 20/页），翻页时保留 `q` 和 `tab` 参数。

性能计时：`time.perf_counter()` 毫秒级显示搜索耗时。

3.6 评论功能

操作	方法	说明
发表评论	POST <code>/song/<id></code>	匿名/实名，底部固定输入栏
查看评论	GET <code>/song/<id></code>	按时间倒序，hover 显示删除按钮
删除评论	POST <code>/comment/<id>/delete</code>	无权限校验（基础版设计）

四、数据模型

```
class Singer(models.Model):  
    id = IntegerField(primary_key=True) # 歌手 ID (自编序号)
```

```

name         = CharField(max_length=200)           # 歌手姓名
image_url    = CharField(max_length=200)           # 歌手图片 URL
desc         = TextField(blank=True)               # 歌手简介
source_url   = URLField(max_length=200)            # 网易云原始链接

class Song(models.Model):
    id         = IntegerField(primary_key=True)     # 歌曲 ID (自编序号)
    name       = CharField(max_length=200)          # 歌曲名
    image_url  = CharField(max_length=200)          # 封面图 URL
    source_url = URLField(max_length=200)           # 网易云原始链接
    lyrics     = TextField(blank=True)              # 歌词 (\n 分隔)
    singers    = ManyToManyField(Singer)           # 歌手 (多对多)

class Comment(models.Model):
    name       = CharField(max_length=200)          # 评论者昵称
    content    = TextField(blank=True)              # 评论内容
    song       = ForeignKey(Song)                   # 所属歌曲
    create_time = DateField()                       # 发布日期

```

五、核心算法

5.1 每日随机排序

```

DAILY_SEED = int(date.today().strftime('%Y%m%d'))
random.seed(DAILY_SEED)
RANDOM_LIST = [random.randint(1, 1000000000) for _ in range(10000)]

# 使用时:
songs = sorted(songs, key=lambda x: RANDOM_LIST[x.id])

```

原理：模块加载时以当天日期为种子，生成一万个预计算随机数。每首歌以 `RANDOM_LIST[id]` 作为排序键——同一 ID 在同一天映射到同一随机数，结果全天一致；跨天后种子变化，顺序重新洗牌。每次请求复用同一序列表，零额外计算开销。

5.2 搜索四级优先级

```

singers = Singer.objects.filter(
    Q(name__icontains=query) | Q(desc__icontains=query)
).annotate(
    priority=Case(
        When(name__iexact=query, then=Value(0)),           # 精确匹配
        When(name__icontains=query, then=Value(1)),        # 模糊匹配
        When(desc__icontains=query, then=Value(2)),        # 简介匹配
        default=Value(3),
        output_field=IntegerField()
    )
).order_by('priority', 'name').distinct()

```

原理：Django ORM 的 `Case/When` 构造 `CASE WHEN ... THEN ... END` SQL，对每条记录标注优先级数字。排序时优先级数字小的在前，同类内部按名称排序。避免在 Python 层手工排序，全部由数据库引擎完成。

5.3 多栏歌词自适应

```
lyrics_len = len(song.lyrics.split("\n"))
lyrics_cols = min(5, (lyrics_len + 29) // 30) # ceil(行数/30), 最多 5 栏
```

原理： `(n + 29) // 30` 等效于 `ceil(n/30)` 的整数运算。在 CSS 中设置 `column-count: N` 和 `column-rule` 实现多栏排版。

六、爬虫模块

代码位于 `crawler/` 目录，四个文件，约 600 行。

6.1 架构

模块	文件	主要功能
请求层	<code>fetcher.py</code>	HTTP GET/POST、自动重试、UA/Cookie 伪装、图片下载
解析层	<code>parser.py</code>	JSON 解析、歌词文本清洗（去除 HTML 标签和元信息行）、歌手信息字段提取
存储层	<code>storage.py</code>	JSON 数据持久化、图片二进制保存、fixture 格式转换（ <code>json.dumps</code> ， <code>ensure_ascii=False</code> ）
编排层	<code>crawler.py</code>	流程编排，定义 <code>LEAST_SONG</code> / <code>LEAST_SINGER</code> 阈值，控制爬取范围和顺序

6.2 核心技术

技术	用途
<code>requests.Session</code>	复用 TCP 连接，维持 Cookie
<code>fake_useragent</code>	每次请求随机切换 User-Agent
<code>time.sleep</code>	随机间隔 1~2 秒，避免触发频率限制
<code>json.loads</code> / <code>json.dumps</code>	网易云 API 响应解析 + 本地 fixture 生成
<code>re.search</code> / <code>re.sub</code>	歌词元信息提取、行尾空白清理
<code>os.makedirs</code> + <code>open('wb')</code>	图片目录创建 + 二进制写入
<code>random.shuffle</code> / <code>random.sample</code>	歌手列表洗牌、hotSongs 随机采样

6.3 关键算法

失败重试机制 (`fetcher.py`) :

```
def fetch(url, retry=8):
    for attempt in range(retry):
        try:
            resp = session.get(url, headers=headers, timeout=10)
            if resp.status_code == 200:
                return resp.json()
        except Exception:
            time.sleep(random.uniform(5, 10)) # 指数退避
    return None
```

最多重试 8 次, 间隔 5~10 秒随机。单次请求超时 10 秒, 避免永久阻塞。

歌曲去重 (`crawler.py`) :

从歌手 `hotSongs` 和歌单两个来源获取歌曲 ID, 使用 Python `set` 去重后, 统一拉取详情。ID 集合的 `|` 并集运算保证 $O(n)$ 去重效率。

数据重编号 (`crawler.py` `rebuild_data()`) :

爬取完成后, 所有歌曲和歌手统一从 1 开始重新分配自增 ID。多对多关系通过临时字典 `{old_id: new_id}` 映射重建, 保持外键引用的完整性。

fixture 生成 (`storage.py`) :

```
# 歌手 fixture 格式: {"model": "web.singer", "pk": id, "fields": {...}}
# 歌曲 fixture 格式: {"model": "web.song", "pk": id, "fields": {..., "singers": [id1, id2]}}
# 评论 fixture 格式: {"model": "web.comment", "pk": 自动递增, "fields": {...}}
```

三个 fixture 文件按 Singer → Song → Comment 顺序 `loaddata`, 确保外键依赖正确。

6.4 爬取流程

1. <code>build_cdlist()</code>	从华语歌单分类中获取 165 个歌单 ID
2. <code>build_sonplist_by_singer()</code>	
— 获取网易云男/女歌手列表 (各 100 位)	
— 随机打乱, 前置彩蛋歌手 (HOYO-MiX, 知更鸟, Chilichill)	
— 从每位歌手的 <code>hotSongs</code> 中随机取 8~11 首	
3. <code>build_sonplist()</code>	合并歌手歌曲 + 歌单歌曲, <code>set</code> 去重
4. <code>build_songs()</code>	批量获取歌曲详情 (每次 10 首, 合并 ID 逗号分隔)
5. <code>build_lyrics()</code>	逐首获取歌词, 清洗 HTML 和元信息
6. <code>rebuild_data()</code>	重新编号 (<code>song.id 1~N</code> , <code>singer.id 1~M</code>), 字典映射重建 M2M
7. <code>build_singers()</code>	获取每位歌手详情、简介、封面图
8. <code>build_comments()</code>	从网易云热评中随机取 8~15 条/首
9. <code>transform_data()</code>	转换为 Django fixture JSON 格式

6.5 反爬策略

策略	实现
----	----

随机 UA	<code>fake_useragent</code> 每次请求生成随机 User-Agent
Cookie 认证	网易云 Cookie 绕过部分 API 限制
请求间隔	<code>Sleep()</code> 函数, 随机间隔 1~2 秒
失败重试	最多 8 次, 间隔 5~10 秒随机 (指数退避)
Referer 伪装	统一设置为 <code>https://music.163.com/</code>
超时控制	<code>requests.get(timeout=10)</code> , 防止单请求永久挂起

6.6 网易云 API 端点

用途	端点
歌单分类	<code>music.163.com/discover/playlist/?cat=华语</code>
歌单详情	<code>music.163.com/api/playlist/detail?id=<id></code>
歌曲详情	<code>music.163.com/api/song/detail?ids=[...]</code>
歌词	<code>music.163.com/api/song/lyric?os=pc&id=<id>&lv=-1</code>
歌手 hotSongs	<code>music.163.com/api/artist/top/song?id=<id></code>
歌手详情	<code>music.163.com/api/artist/desc?id=<id></code>
歌手专辑	<code>music.163.com/api/artist/albums/<id></code>
评论	<code>music.163.com/api/v1/resource/comments/R_SO_4_<song_id></code>

七、数据分析模块

代码位于 `analysis/` 目录, 三个子模块, 共 6 个脚本, 配套 3 份分析报告。

7.1 模块概览

子模块	目录	核心问题	脚本数
最佳拍档	<code>matches/</code>	谁跟谁合作最紧密?	2
常用词	<code>lyrics/</code>	谁用大众词最多?	3
独特词	<code>special/</code>	谁的词汇最不可替代?	2

7.2 使用技术

技术	用途	所在模块
<code>jieba</code>	中文分词（精确模式）	常用词 / 独特词
<code>numpy</code>	数组运算、中位数、指数函数 <code>exp</code>	独特词（坐标变换）
<code>matplotlib</code>	柱状图、散点图、聚类树状图	全部
<code>wordcloud</code>	歌手个人词云图生成	常用词
<code>json</code>	读取 fixture 数据	全部
<code>re</code>	正则清洗歌词元信息	全部
<code>math.log</code>	TF-IDF 的 IDF 计算	独特词
<code>pathlib.Path</code>	跨平台路径拼接	全部

7.3 最佳拍档算法

核心数据结构：

```
matches = {} # {(singer_id, writer_id): count}
for song in all_songs:
    for singer in song.singers:
        for writer in song.writers:
            if singer != writer: # 排除自己作词自己唱
                matches[(singer, writer)] += 1
```

三重循环遍历每首歌的歌手列表 × 作词列表，用 `(id1, id2)` 元组做字典键， $O(n \times s \times w)$ 统计共现次数。去除了搭档两人为同一人的情况。

可视化：`matplotlib` 横向柱状图，Top 30 渐变着色（前 10 紫色、中间 10 红色、后 10 橙色）。

7.4 常用词算法

jieba 分词 + 词频统计：

```
# 对全部 4,718 首歌词逐首分词
seg_list = jieba.cut(lyrics)
for word in seg_list:
    if is_chinese_word(word) and word not in ban_words:
        word_count[word].Add(song.singers, song.writers)
```

`is_chinese_word` 用正则 `[一-隹...]+` 过滤非中文词和单字词。`ban_words` 排除 27 个音乐制作术语（“作词”“作曲”“编曲”等）。

Word 类：每个高频词维护两个字典——`self.singers = {singer_id: count}` 和 `self.writers = {writer_id: count}`，记录该词被哪些歌手/作词人使用、各用了多少次。最终按总频次 > 500 筛选出 119 个常用词。

词云图 (`build_wordcloud.py`) :

```
# 对每位歌手, 收集其使用的所有词及频次
word_dict = {word: count for word, data in sorted_word
              if data.singers.get(singer_id, 0) >= 5}
WordCloud(font_path=..., width=800, height=400, scale=2,
          background_color='white').generate_from_frequencies(word_dict)
```

筛选阈值 ≥ 5 次以过滤噪音词。 `scale=2` 提升输出分辨率。

7.5 独特词算法 (TF-IDF)

TF 计算: 对每位歌手, 将其全部歌词拼接为一个文档。

```
tf[singer][word] = count_of_word / total_words_of_singer
```

IDF 计算:

```
idf[word] = math.log(total_singers / singers_using_word)
```

TF-IDF: `tfidf(singer, word) = tf * idf`。每个词在特定歌手那里越频繁 (高 TF), 同时在别的歌手那里越少见 (高 IDF), 得分越高。

过滤条件:

- 中文词 (≥ 2 字)
- 非制作术语 (`ban_words`)
- 非歌手/作词人名 (防止同名干扰)
- 全局出现 ≥ 10 次 (排除极端冷僻词)
- 过滤含标点行 (`:`、`/()`、`()`), 去除作词/作曲等元信息行)

评分: 每位歌手取 Top 10 标志词的 TF-IDF 之和作为总分。

7.6 四象限散点图算法

坐标变换:

```
# Y 轴:  $1 - e^{(-18x)}$ , 底部线性拉伸, 顶部饱和
trans_y(x) = 10 * (1 - np.exp(-18 * x))

# X 轴: 分段线性, 中间人群斜率最大, 两端压缩
trans_x(x):
    if 500 <= x <= 1750: return 2000 + (x-500)*6 # 斜率 6
    elif x > 1750:       return 9500 + (x-1750)*2 # 斜率 2
    else:                return x*4                # 斜率 4
```

原理: Y 轴使用 `$1 - e^{(-kx)}$` 函数, $x \rightarrow 0$ 时近似 kx (线性区分度好), x 增大时自动饱和至上限, 防止游戏厂牌的极高 TF-IDF 值压缩中部的视觉空间。X 轴分段线性使中间密集人群 (500~1750) 占据最多视觉面积。象限线使用中位数分割。

7.7 分析结论对应

分析	报告	关键结论
最佳拍档	总结 1.md	陶喆&娃娃 37 次合作登顶；游戏音乐工业化生产；C-Block 团体创作模式
常用词	总结 2.md	法老双榜第一；姚若龙/林夕/李焯雄三巨头；李荣浩"一样"等 6 词被垄断
独特词	总结 3.md	游戏配乐词汇最封闭；王菲佛经独一；四象限：全能 9/纯独特 16/纯常用 16/双低 8

三篇汇总见 `analysis/实验报告.md`。

八、URL 路由

路径	视图	说明
<code>/</code>	<code>index</code>	首页，重定向到歌曲列表
<code>/songlist</code>	<code>songlist</code>	歌曲列表（分页，每页 20 首）
<code>/singerlist</code>	<code>singerlist</code>	歌手列表（分页，每页 25 位）
<code>/song/<id></code>	<code>song</code>	歌曲详情 + 评论区 + 评论提交
<code>/singer/<id></code>	<code>singer</code>	歌手详情
<code>/search?q=&tab=</code>	<code>search_view</code>	搜索（tab=singer/song）
<code>/comment/<id>/delete</code>	<code>delete_comment</code>	删除评论

九、技术栈

层级	技术
后端框架	Django 6.0
数据库	SQLite3
前端	Django Templates + 原生 CSS
字体	HarmonyOS Sans SC（Black + Regular）
爬虫 HTTP	<code>requests</code>

层级	技术
UA 伪装	<code>fake_useragent</code>
HTML 解析	BeautifulSoup4 (依赖已安装, 实际未使用)
图片存储	本地 <code>data/images/</code> , 生产走 GitHub raw CDN
中文分词	<code>jieba</code> (精确模式)
数值计算	<code>numpy</code> (<code>exp</code> 、 <code>median</code> 、数组运算)
可视化	<code>matplotlib</code> (柱状图、散点图)
词云	<code>wordcloud</code>
聚类	<code>scipy.cluster.hierarchy</code> (Ward 层次聚类)

十、部署

支持部署到 PythonAnywhere (免费账户) :

```
# WSGI configuration
import os, sys
path = '/home/<username>/task_singers'
if path not in sys.path:
    sys.path.insert(0, path)
os.environ['DJANGO_SETTINGS_MODULE'] = 'config.settings'
os.environ['IMAGE_BASE_URL'] = 'https://raw.githubusercontent.com/.../images/'
from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()
```

图片通过 `IMAGE_BASE_URL` 环境变量指向 GitHub raw, 不占 PA 存储。 `context_processors.image_base` 将其注入所有模板上下文。